

Week 3

Game Engine Development Unity 2D Game Development

Hooman Salamat

Objective & Concept

Objective

- Get started with 2D game development

Concepts

- You can use the Unity Editor to create projects in 3D and 2D. Read this section to learn how to develop 2D projects in the Editor, including how to use sprites, create Tilemaps, and use the 2D physics system.

Introduction to 2D

Many areas of Unity's documentation apply to both 2D and 3D development.

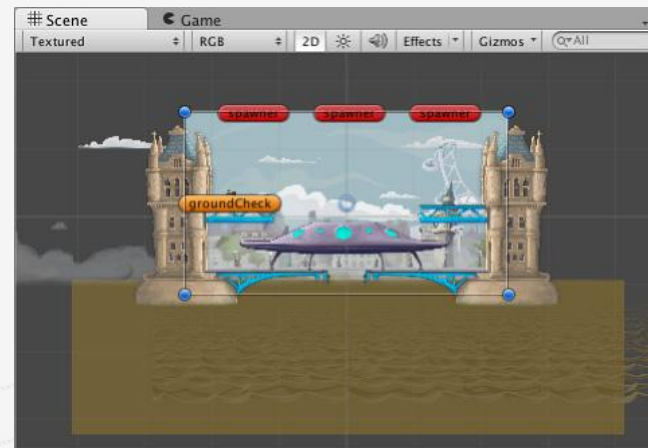
This section focuses on 2D-specific features and functionality.

Gameplay in 2D

The familiar functions of the Unity Editor when you use 3D mode are still available but with helpful additions to simplify 2D development.

The most noticeable feature is the **2D** view mode button in the **toolbar** of the **Scene** view. When you enable 2D mode, Unity will set an orthographic (perspective-free) view.

This view means that the **camera** looks along the z-axis with the y-axis increasing upwards, which enables you to visualize the scene and easily place **2D objects**.



2D graphics

Graphic objects in 2D are known as **Sprites**.

Sprites are standard textures, with special techniques for combining and managing **sprite** textures for efficiency and convenience during development.

Unity provides a built-in [Sprite Editor](#) to let you extract sprite graphics from a larger image.

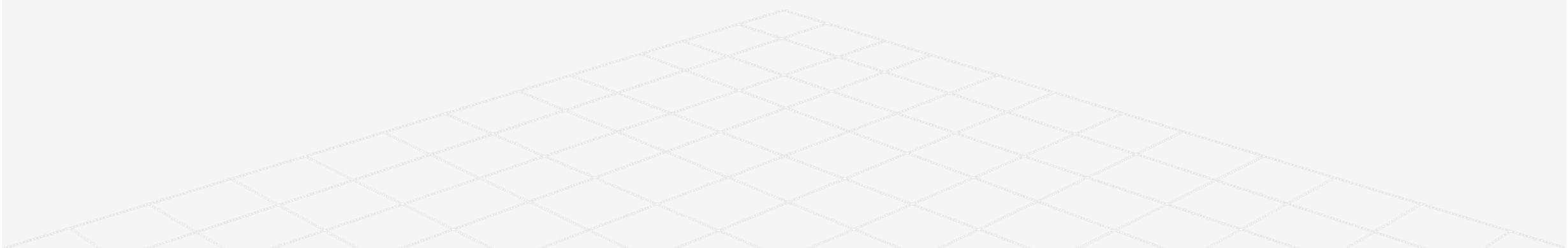
This enables you to edit many component images within a single texture in your image editor.

You can use this, for example, to keep the arms, legs, and body of a character as separate elements within one image.

Sprites are rendered with a [Sprite Renderer](#) component rather than the [Mesh Renderer](#) used with 3D objects.

You can add this to a GameObject via the Components menu (**Component** > **Rendering** > **Sprite Renderer**) or you can create a GameObject directly with a Sprite Renderer already attached (menu: **GameObject** > **2D Object** > **Sprite**).

You can also use a [Sprite Creator](#) tool to make placeholder 2D images.



2D physics

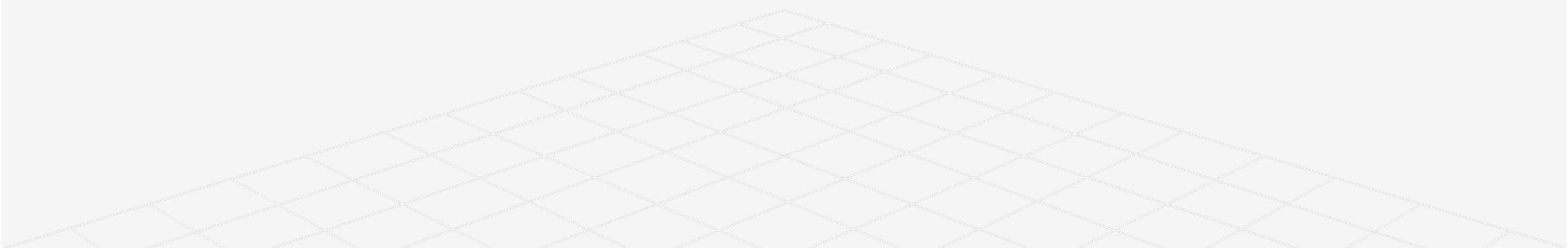
Unity has a separate physics system to handle 2D physics so you can make use of optimizations only available with 2D.

The components correspond to the standard 3D physics components such as [Rigidbody](#), [Box Collider](#), and [Hinge Joint](#), but with 2D appended to the name.

You can equip sprites with [Rigidbody 2D](#), [Box Collider 2D](#), and [Hinge Joint 2D](#).

Most 2D physics components are flattened versions of the 3D equivalents (for example, [Box Collider 2D](#) is a square while [Box Collider](#) is a cube) but there are a few exceptions.

For the full list of 2D physics components, refer to [2D or 3D Projects](#).



Set up a project for 2D games with the Universal 2D template

Create a project with the Universal 2D template. The template automatically installs the [2D feature set](#) collection of packages, giving you a suite of tools and features specifically designed to support 2D game development.

Follow these steps:

1. [Create a new project that uses URP](#). Select the Universal 2D template.
2. Install [optional packages](#) you need for your project.

The following are the packages included by default when you select the Universal 2D template.

Package	Description
2D Animation	This package includes features and tools to rig your 2D characters and sprites for skeletal animation.
2D PSD Importer	This package allows you to import multilayered PSD files from Adobe Photoshop. You can use this for your sprites, or to rig your characters.
2D Aseprite Importer	This package is an asset importer that imports Aseprite's .ase and .aseprite files into Unity.
2D Sprite	This package provides the Sprite Editor to create and edit sprite assets.
2D SpriteShape	This package allows you to create organic shapes and terrain, similar to a vector drawing tool.
2D Tilemap Editor	This package allows you to create grid-based worlds with square, hexagonal or isometric tiles. Add your tiles to the tile palette, and paint and fill tile grids using different settings and Brushes.
2D Tilemap Extras	This package contains extra scripts for use with the 2D Tilemap Editor package, including custom tiles and Brushes.

2D game creation workflow

To create a 2D game, [set up your Unity project](#) and then familiarize yourself with the relevant concepts in the following order:

[Fundamentals](#)

[Scripting](#)

[Sprites](#)

[Building in-game environments](#)

[Character animation](#)

[Graphics](#)

[Physics 2D](#)

[Audio](#)

[User interface](#)

[Accessibility](#)

[Profiling, optimizing and testing](#)

[Publishing](#)

Fundamentals

GameObjects are fundamental objects in Unity that represent characters, props, scenery, and more. Every object in your game is a GameObject.

GameObjects represent the items in your game; the space in which you place them to build your level is called a scene.

Scenes in Unity are always 3D; when you make a 2D game in Unity, you typically choose to ignore the third dimension (the z-axis) but you can also use it in special cases, for example when making 2.5D games.

The behavior of GameObjects is defined by blocks of functionality called components. The following components are fundamental for 2D games:

Transform: the Transform component determines the Position, Rotation, and Scale of each GameObject in the scene. Every GameObject has a Transform component.

Sprite Renderer: the Sprite Renderer component renders the Sprite and controls how it looks in a scene.

Cameras: devices that capture and display the world to the player. Marking a Camera as Orthographic removes all perspective from the Camera's view. This is mostly useful for making isometric or 2D games.

Collider 2D: this component defines the shape of a 2D GameObject for the purposes of physical collisions.

Scripting

All 2D games need [scripts](#).

Scripts respond to input from the player and arrange for events in the gameplay to happen when they should.

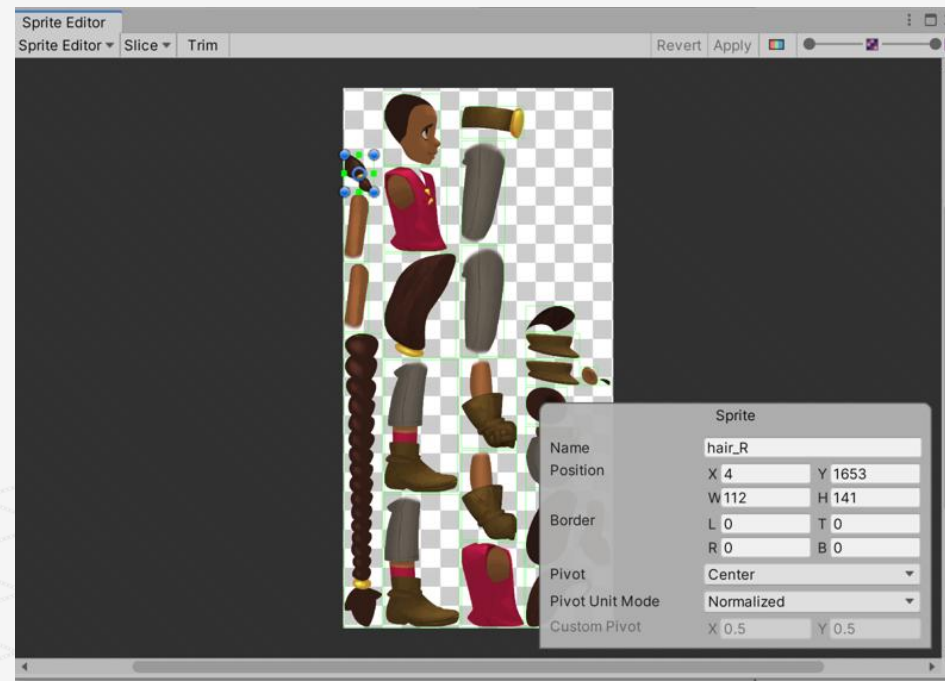
For details on how to use scripts in Unity, refer to [Programming in Unity](#) and the Unity Learn [Beginner Scripting](#) course.

Scripts are attached to GameObjects, and any script you create inherits from the [MonoBehaviour](#) class.

Sprites

Sprites are 2D graphic objects. You use Sprites for all types of 2D games. For example, you can import an image of your main character as a Sprite.

You can also use a collection of Sprites to build a character. This allows you greater control over the movement and animation of your characters.



Import a sprite or spritesheet texture

To use an imported image as one or more sprites, set its texture type to Sprite (2D and UI).

Follow these steps:

Make sure the 2D Sprite package is installed in your project. For more information, refer to [set up your project for 2D games](#).

Import your sprite image or spritesheet. For example, drag a spritesheet .png file into the Project window.

Select the imported image in the Project window.

In the Inspector window, set Texture Type to Sprite (2D and UI).

Select Apply.

Note: In a 2D project, Unity automatically sets Texture Type to Sprite (2D and UI) when you import an image.

Divide the texture into sprites

The Sprite Mode property of the texture determines whether Unity creates one sprite or multiple sprites from the texture.

For each sprite, Unity creates a sprite asset () as a child of the texture in the Project window.

Set Sprite Mode to the following:

Single to use the texture as one sprite, either using the whole image or part of the image.

Multiple to divide the texture into multiple sprites, for example if the texture is a spritesheet.

Add a sprite to a scene

To add a sprite to a scene, drag the sprite asset () from the Project window into the Scene window or Hierarchy window.

Unity creates a GameObject with a Sprite Renderer component. The component renders the sprite and controls how it appears in the scene.

Add a sprite to an existing GameObject

To add a sprite to an existing GameObject, follow these steps:

Select the GameObject in the Scene or Hierarchy window.

In the Inspector window, select Add Component.

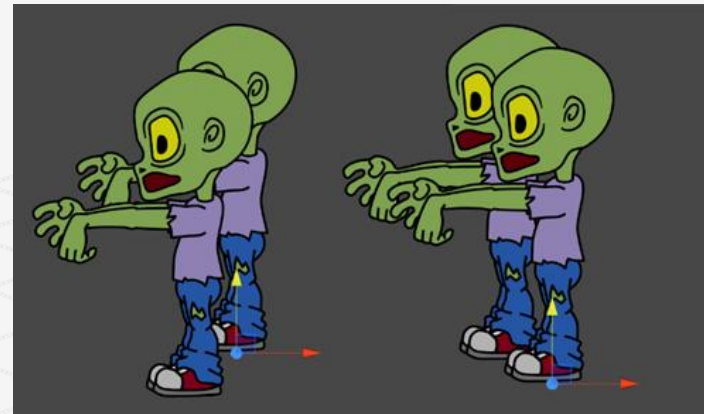
Select Rendering > Sprite Renderer.

In the Sprite Renderer component, set the Sprite property to the sprite you want to render.

Rendering Sprites

Use the [Sprite Renderer](#) component to render your Sprites. For example, you can use the Sprite Renderer to change the color and opacity of a Sprite.

Sorting Sprites By organizing Sprites in layers, you can create an illusion of depth. You can sort Sprites according to many strategies.

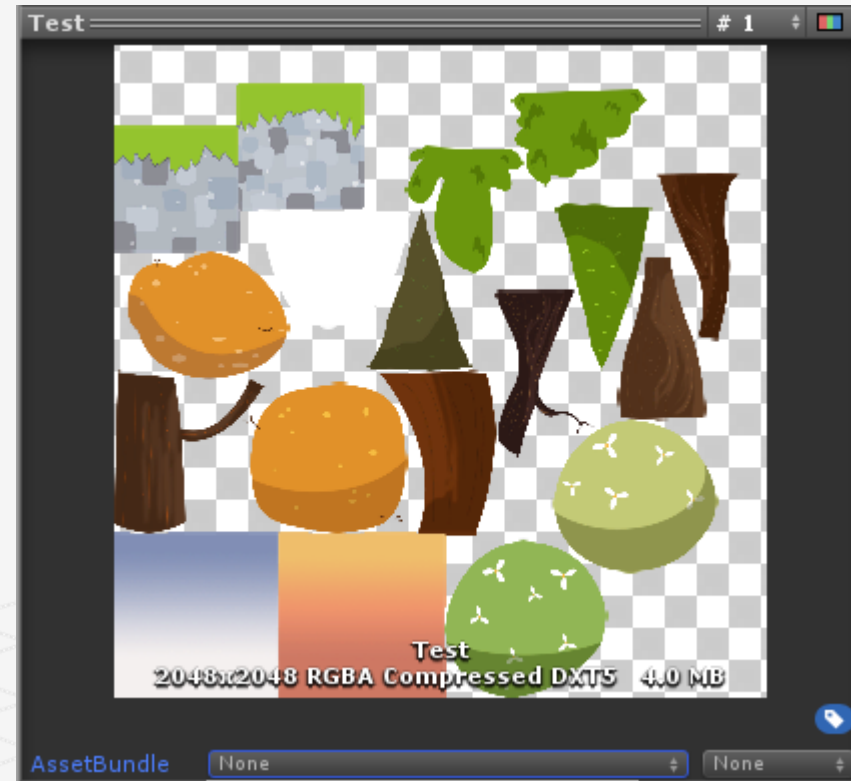


Sprite Atlas

You can use a [Sprite Atlas](#) to consolidate several Textures into a single combined Texture.

This optimizes your game and saves memory.

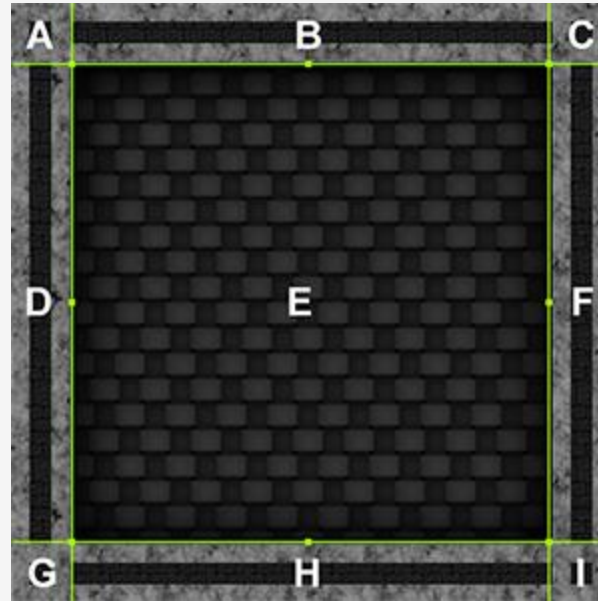
For example, you can add all your Sprites associated with a particular character or purpose to a Sprite Atlas.



9-slicing

9-slicing is a 2D technique that allows you to reuse an image at various sizes without needing to prepare multiple assets. Unity can dynamically stretch and tile designated parts of a Sprite to allow one Sprite to serve as the border or background for UI elements of many sizes. Refer to [9-slicing Sprites](#).

For example, you could use 9-slicing to stretch a Sprite to shape when you build a 2D level.

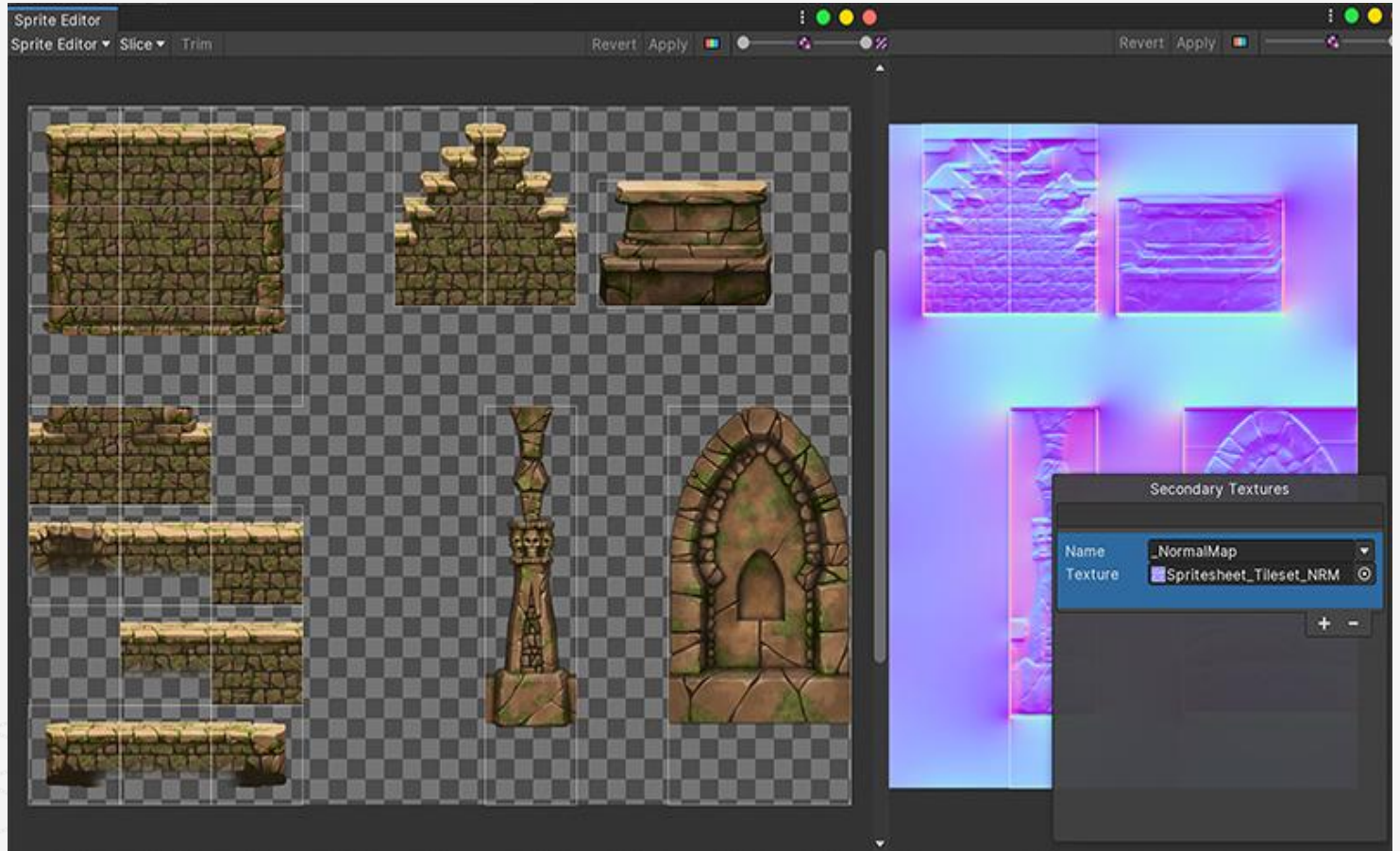


Tilemap

The [Tilemap](#) component is a system that stores and handles Tile assets for creating 2D levels.

Use the [2D Tilemap Editor](#) package (installed by default) to use Tilemaps.

For example, you can use Tilemaps to paint levels using Tiles and brush tools and define rules for how Tiles behave.



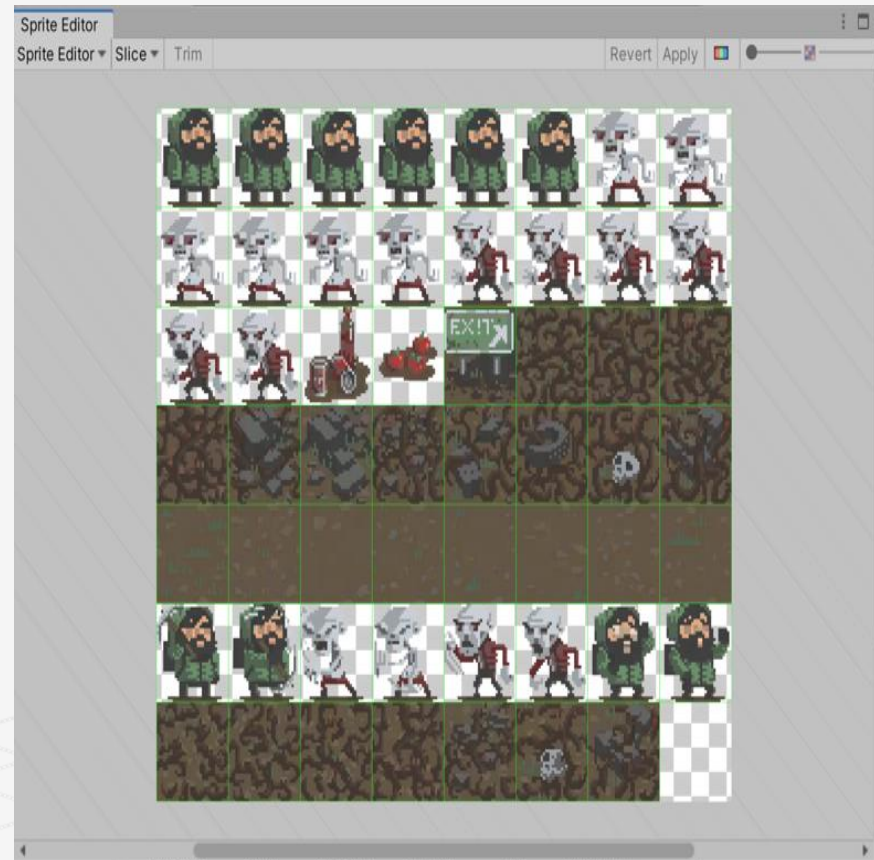
Character animation

There are three different ways you can animate 2D characters:

2D animation type	Used for
Frame-by-frame	Artistic reasons, if you want your game to have a classic animation art style. Frame-by-frame animation is relatively resource-intensive, both to make and to run.
Cutout	Smooth skeletal animation, when the characters don't require realistic articulation.
Skeletal	Smooth skeletal animation where Sprites bend according to the bone structure. Use this when the characters need a more organic feel.

Frame-by-frame

Frame-by-frame animation is based on the traditional cel animation technique of drawing each moment of an animation as individual images, which are played in fast sequence, like flipping pages on a flipbook.



Cutout

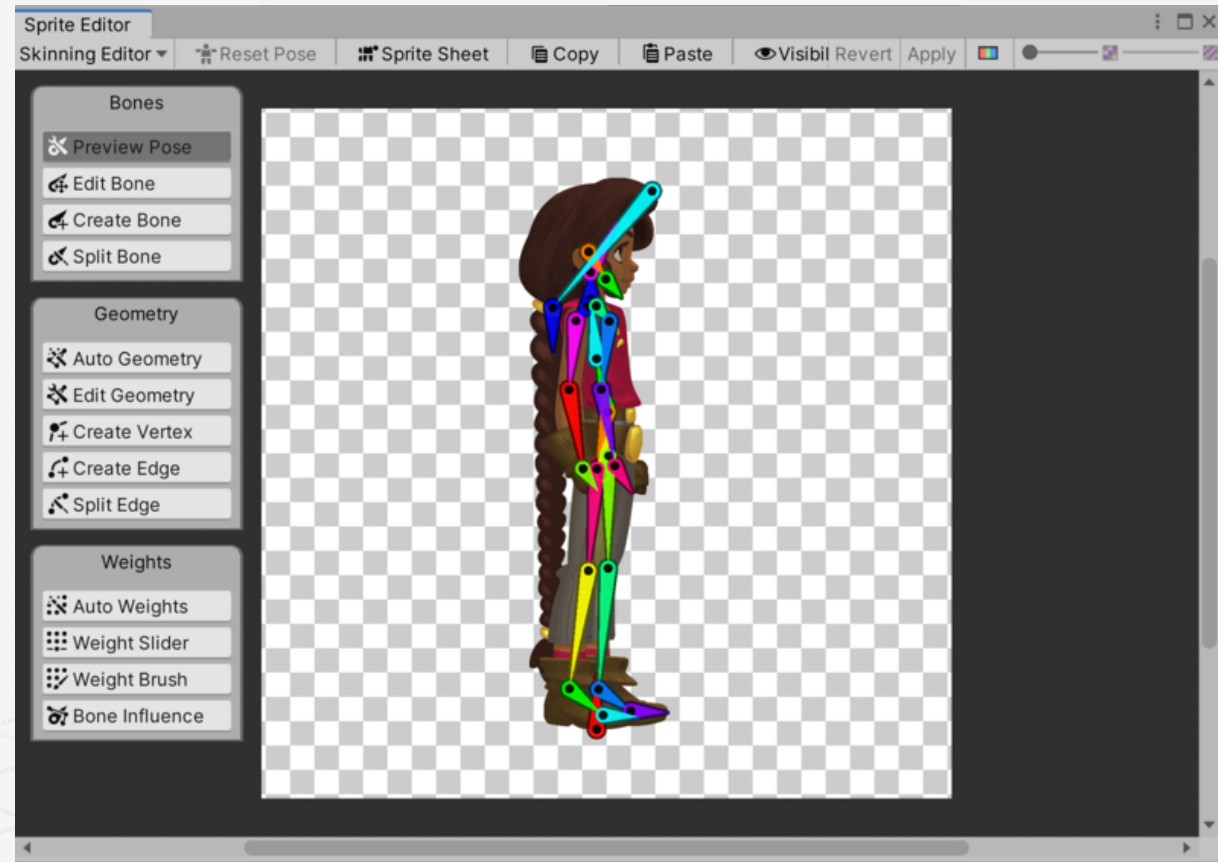
In cutout animation, multiple Sprites make up the body of a character, and each piece moves to give the visual effect of the whole character moving. This animation style is similar to skeletal animation, except that the Sprites don't bend.



Skeletal

With skeletal animation, you map a Sprite or a group of Sprites onto an animation skeleton. You can create and define animation bones for characters and objects, that define how they should bend and move. This approach allows the bones to bend and deform the Sprites, for a more natural movement style. To use skeletal animation, you need to use the [2D Animation](#) package (installed by default).

For a 2D Animation workflow, including a guide to working with the Bone Editor, refer to the [2D Animation documentation](#).



Lighting

Because you're using URP with the 2D Renderer, you can use the Light 2D component to apply optimized 2D lighting to Sprites. For details, refer to [Introduction to Lights 2D](#).

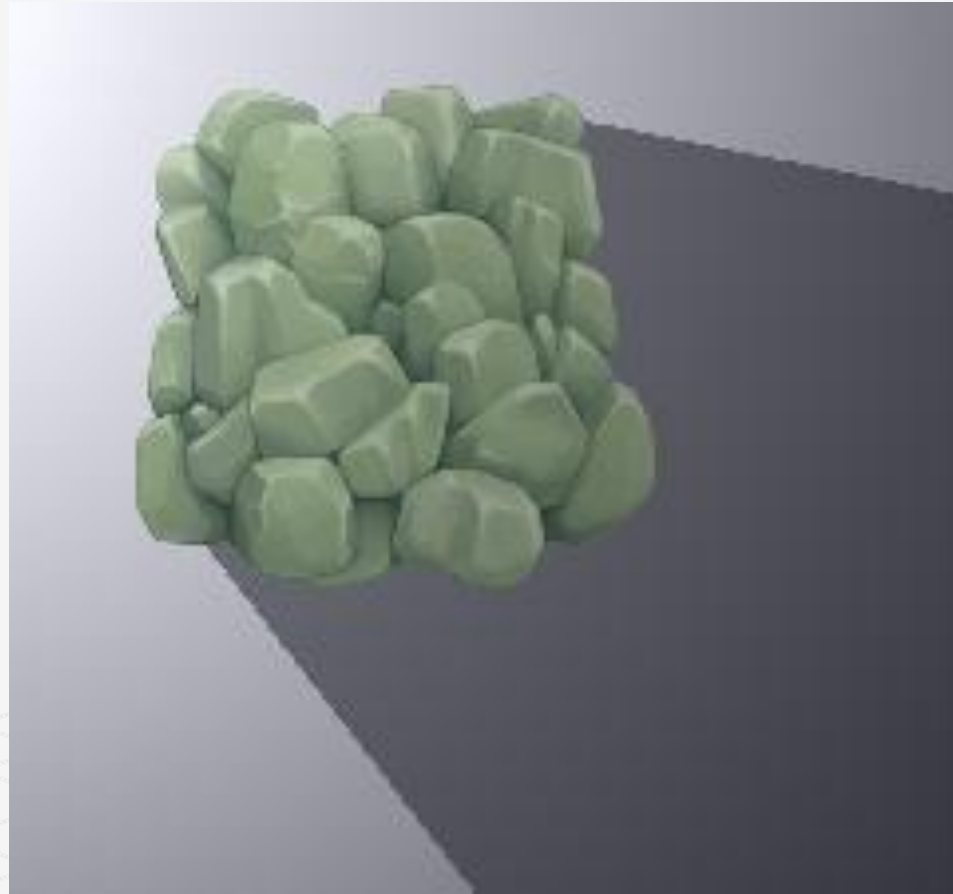
These two images show the same scene; in the image on the left, 2D Lights are disabled, and in the image on the right, 2D lights are enabled. With 2D Lights, you can use the same Sprites to create different weather conditions or moods.



Shadows

To define the shape and properties that a Light uses to determine the shadows it casts, use the [Shadow Caster 2D component](#).

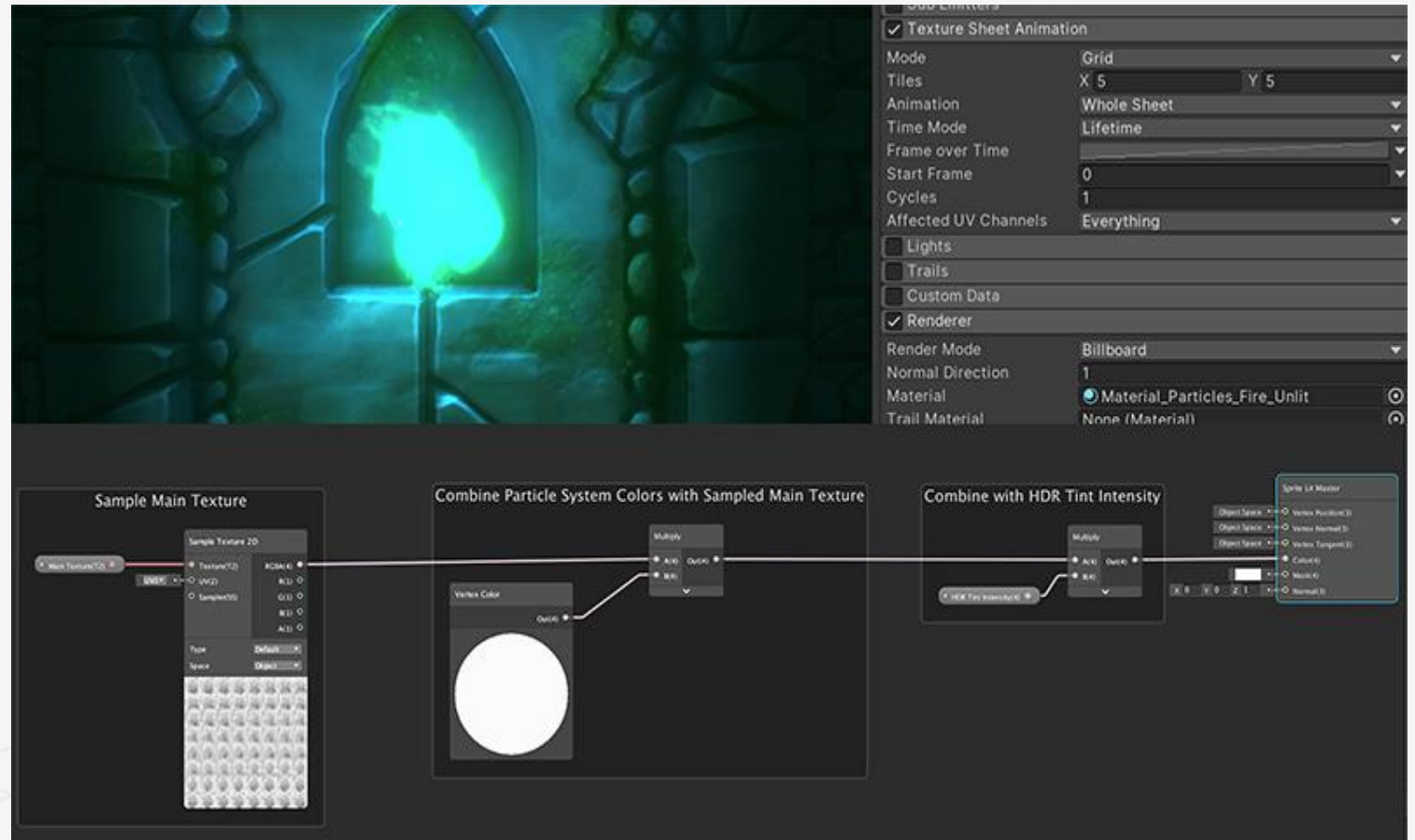
A shadow intensity of 0.5 in the Shadow Caster 2D component.



Particle systems

You can use particle systems to create dynamic objects like fire, smoke or liquids, as an alternative to using a Sprite. Sprites are more suited to physical objects. Refer to [Particle systems](#)

A fire effect, created with the Particle System and Shader Graph for 2D.

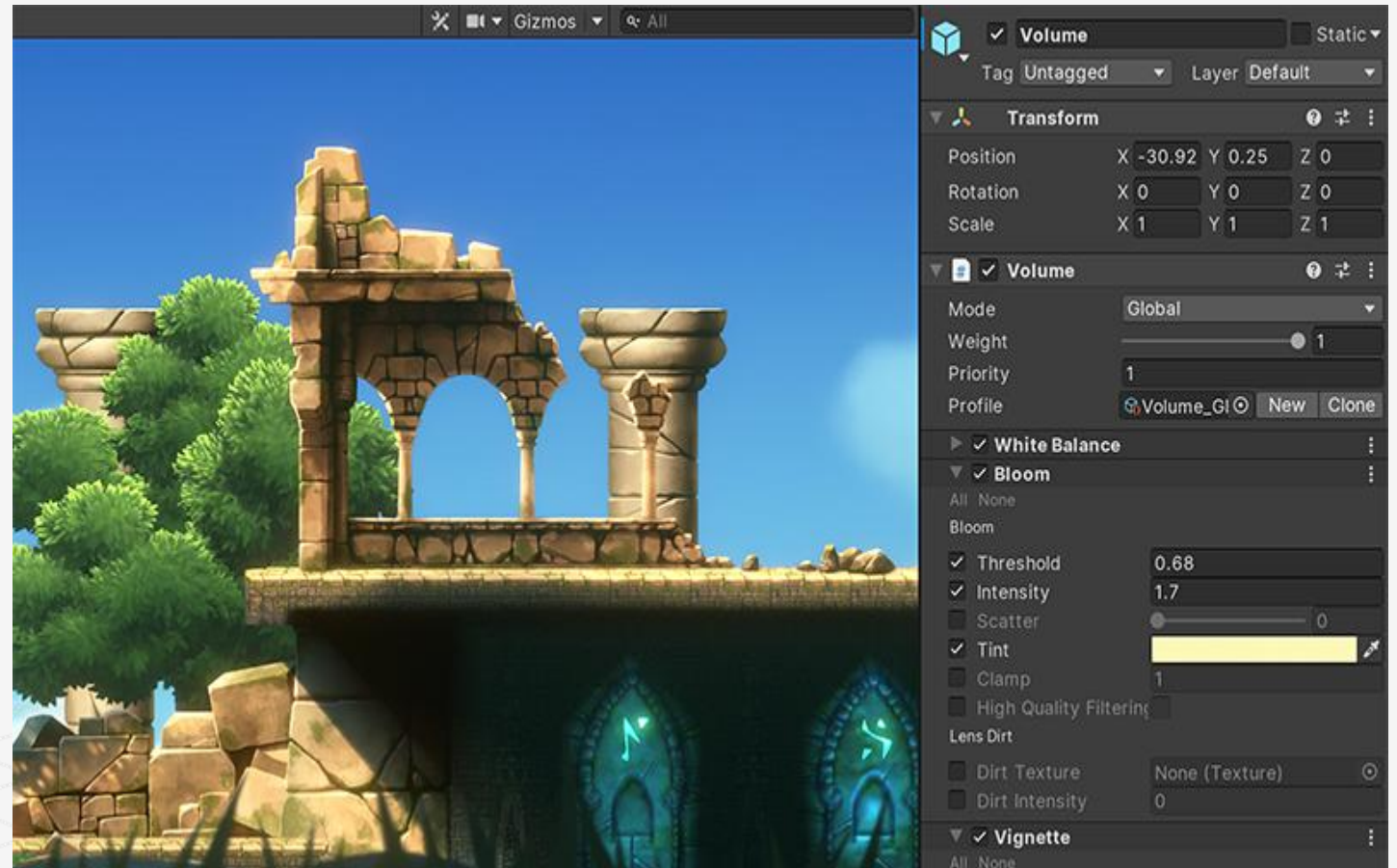


Post-processing

You can use post-processing effects and full-screen effects to significantly improve the appearance of your game. For example, you can use these effects to simulate physical camera or film properties, or to create stylized visuals.

URP has its own post-processing implementation. Refer to [Post-processing in the Universal Render Pipeline](#).

The Lost Crypt demo uses the bloom and vignette post-processing effects



Physics 2D

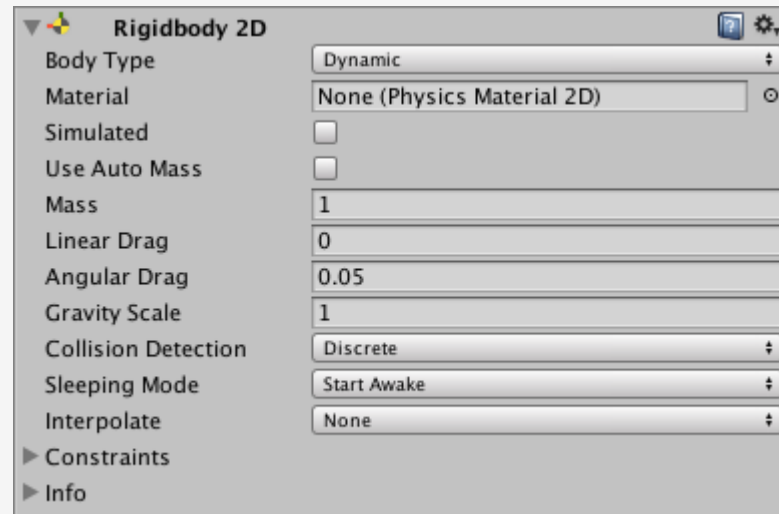
To learn how to use Unity's 2D physics engine, refer to [Applying 2D Colliders for Physics Interactions](#).

The following 2D physics tools are useful for 2D games.

- ❑ Rigidbody 2D
- ❑ Collider 2D
- ❑ Triggers
- ❑ 2D Joints
- ❑ 2D Effectors

Rigidbody 2D

A Rigidbody 2D component places a GameObject under the control of the physics engine. Refer to [Rigidbody 2D](#).



Collider 2D

Collider 2D components define the shape of a 2D GameObject for the purposes of physical collisions. You can also use Collider 2D components for input detection.

For example, in mobile games you can use them to make Sprites selectable.

The Collider 2D types that you can use with Rigidbody 2D are:

[Circle Collider 2D](#)

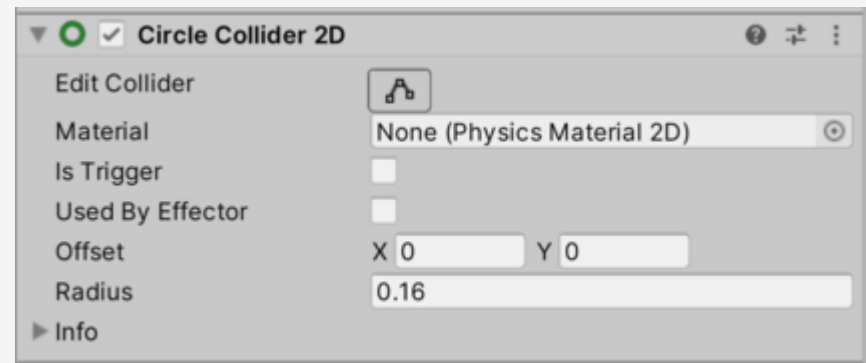
[Box Collider 2D](#)

[Polygon Collider 2D](#)

[Edge Collider 2D](#)

[Capsule Collider 2D](#)

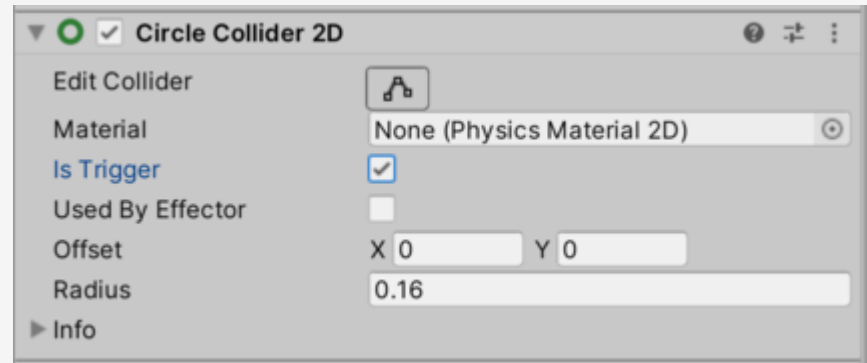
[Composite Collider 2D](#)



Triggers

When you set a Collider 2D as a Trigger (by enabling its **Is Trigger** property), it no longer behaves as a physical object, and it can intersect with other Colliders without causing a collision.

Instead, when a Collider enters its space, Unity calls the `OnTriggerEnter` function on the Trigger GameObject's scripts.



2D Joints

Joints attach GameObjects together. You can only attach 2D joints to GameObjects that have a Rigidbody 2D component attached, or to a fixed position in world space. Refer to [2D Joints](#).

Page	Description
Introduction to 2D joints	Understand the fundamental concepts of 2D joints.
2D joint constraints	Understand the constraints of using 2D joints.
Distance Joint 2D	Techniques and resources for working with Distance Joint 2D components.
Fixed Joint 2D	Techniques and resources for working with Fixed Joint 2D components.
Friction Joint 2D	Techniques and resources for working with Friction Joint 2D components.
Hinge Joint 2D	Techniques and resources for working with Hinge Joint 2D components.
Relative Joint 2D	Techniques and resources for working with Relative Joint 2D components.
Slider Joint 2D	Techniques and resources for working with Slider Joint 2D components.
Spring Joint 2D	Techniques and resources for working with Spring Joint 2D components.
Target Joint 2D	Techniques and resources for working with Target Joint 2D components.
Wheel Joint 2D	Techniques and resources for working with Wheel Joint 2D components.

2D Effectors

Use Effector 2D [components](#) with [Collider 2D](#) components to direct the forces of [physics](#) in your scene when [GameObject](#) Colliders come into contact with each other. Refer to [2D Effectors](#).

Page	Description
Area Effector 2D reference	Explore the properties of the Area Effector 2D used to arbitrarily vary force and angle magnitude.
Buoyancy Effector 2D reference	Explore the properties of the Buoyancy Effector 2D used to simulate buoyancy, fluid flow and fluid drag.
Point Effector 2D reference	Explore the properties of the Point Effector 2D used to attract or repulse against a given source point.
Platform Effector 2D reference	Explore the properties of the Platform Effector 2D used to create platform behavior, such as one-way collisions.
Surface Effector 2D reference	Explore the properties of the Surface Effector 2D used to create conveyor belts.

Audio

A game would be incomplete without some kind of audio, be it background music or sound effects. Unity's audio system can import most standard audio file formats, play sounds in 3D space, and apply optional effects such as echo and filtering. Unity can also record audio from any available microphone on your machine for use during gameplay or for storage and transmission.

Basic theory

In real life, objects emit sounds that listeners hear. The way a sound is perceived depends on many factors. A listener can tell roughly which direction a sound is coming from and may also get some sense of its distance from its loudness and quality. A fast-moving sound source (such as a falling bomb or a passing police car) changes in pitch as it moves as a result of the Doppler Effect. Surroundings also affect the way sound is reflected. A voice inside a cave has an echo, but the same voice in the open air doesn't.

Audio Sources and Listener

To simulate the effects of position, Unity requires sounds to originate from Audio Sources attached to objects. The sounds emitted are then picked up by an Audio Listener attached to another object, most often the main camera. Unity can then simulate the effects of a source's distance and position from the listener object and play them to you accordingly. You can also use the relative speed of the source and listener objects to simulate the Doppler Effect for added realism.

Unity can't calculate echoes purely from scene geometry, but you can simulate them by adding Audio Filters to objects. For example, you could apply the Echo filter to a sound that is supposed to be coming from inside a cave. In situations where objects can move in and out of a place with a strong echo, you can add a Reverb Zone to the scene. For example, your game might involve cars driving through a tunnel. If you place a reverb zone inside the tunnel, the cars' engine sounds start to echo as they enter. The echo quiets as the cars emerge from the other side.

With the Unity Audio Mixer, you can mix various audio sources, apply effects to them, and perform mastering.

Working with audio assets

Unity can import audio files in AIFF, WAV, MP3, and Ogg formats in the same way as other assets. Drag the files into the Project panel. Import an audio file to create an Audio Clip that you can then drag to an Audio Source or use from a script. The Audio Clip reference page has more details about the import options available for audio files.

For music, Unity also supports tracker modules, which use short audio samples as "instruments" that you can arrange to play tunes. You can import tracker modules from .xm, .mod, .it, and .s3m files and use them the same way you use other audio clips.

Audio recording

Unity can access the computer's microphones from a script and create Audio Clips by direct recording. The Microphone class provides an API to find available microphones, query their capabilities, and start and end a recording session. The script reference page for Microphone has further information and code samples for audio recording.

User interface

Topic	Description
Introduction to UI Toolkit	Learn about UI Toolkit, its features, roles, and responsibilities in the development process.
Get started with UI Toolkit	Follow a simple workflow to get started with UI Toolkit.
UI Builder	Use a visual authoring tool to create and edit UI Toolkit assets such as UXML and USS files.
Structure UI	Structure your UI with either UXML or C#.
Style UI	Style your UI with USS.
Control behavior with events	Map user interactions to elements, such as input, touch and pointer interactions, drag-and-drop operations, and other event types. This system includes a dispatcher, a handler, a synthesizer, and a library of event types.
UI Renderer	Use a rendering system built directly on top of Unity's graphics device layer.
Data binding	Link properties to the controls that modify their values.
Support for Editor UI	A set of components to create Editor UI.
Support for runtime UI	A set of components to create runtime UI.
Work with text	Learn how to work with text in UI Toolkit, including fonts, styles, and text elements.
Test UI	Learn how to test and debug your UI.
UI Toolkit examples	A collection of UI Toolkit examples to help you learn how to use it.
Migration guides	Guides to help you migrate from uGUI or IMGUI to UI Toolkit.

Accessibility

Implement accessibility features in Unity to create inclusive games and applications for users of all abilities.

When you implement accessibility features, you enable people with disabilities to engage with your content while also improving the user experience for everyone.

Topic	Description
Introduction to the Accessibility module	Learn about Unity's Accessibility module, its purpose, capabilities, and platform support.
Get started with screen reader support	Use the screen reader support APIs to make your first accessible button.
The architecture and performance of the Accessibility module	Discover the architecture of the Accessibility module, including its core components, key concepts, lifecycle, and performance considerations.
Accessibility concepts	Learn the background concepts that underpin accessible development, including disability categories, assistive technologies, and how screen readers work.

Profiling, optimizing and testing a build

Connect your application to the Profiler to measure its performance.

The way you connect to the Profiler depends on the platform you want to profile your application on.

Topic	Description
Collecting performance data introduction	Understand the various ways of collecting performance data through the Profiler.
Collect performance data on a target platform	Connect the Profiler to a device that's running your application.
Collect performance data in Play mode	Profile in Play mode in the Unity Editor.
Collect performance data about the Unity Editor	Profile the Editor process.
Build Profiles Profile settings reference	Adjust how the Profiler connects to your application when you build it.

Testing

Unity Test Framework overview

The Unity Test Framework allows you to test code in both Edit Mode and Play Mode, and also on target platforms such as Standalone, Android, iOS, etc.

This package provides a standard test framework for Unity users and developers so that both benefit from the same features and can write tests the same way.

UTF includes a custom integration of NUnit (based on NUnit 3.5), the open-source unit testing library for .Net languages, adapted to work with Unity. For more information about NUnit, see the official NUnit website and the NUnit documentation.

Using this documentation

The Manual outlines key features, concepts, and workflows for creating and running tests via the Test Runner window. The Scripting API contains descriptions and code examples for the public APIs. You can browse the Scripting API freely or review the features in the Manual to discover what API members you might be interested in.

Prerequisite knowledge

There are some concepts which this documentation assumes you're familiar with:

Assemblies

Assemblies are an important foundational concept in Unity Test Framework. We group tests into Test Assemblies and configure the assembly definitions to reference platforms the tests should run on and to selectively reference the project assemblies we want to test. You should be familiar with the documentation on Assembly Definitions in Unity and how to configure them via the Inspector or, if you prefer, by editing assembly definition (.asmdef) JSON files directly.

NUnit

UTF includes a custom version of NUnit adapted to work with Unity. It's important to note that a lot of the APIs you use when writing UTF tests are NUnit APIs. Some of your tests may not need to use any of the Unity-specific APIs at all. Before you try using UTF you should familiarize yourself with NUnit and you should use the UTF and NUnit documentation side by side.

TDD and unit testing

This documentation is not intended as a general introduction to test driven development (TDD) or unit testing. Although UTF is intended for use by our users as well as our developers, we assume some familiarity with unit testing conventions in C# or a similar language such as:

The Arrange, Act, Assert (AAA) pattern whereby we set up tests, perform an action, and expect a result.

Using attributes to annotate methods and designate them to run at different phases of the test execution.

Implementing interfaces and callbacks.

See the Microsoft documentation for an introduction to unit test fundamentals in Visual Studio, much of which is similar to the way we create, write, and run UTF tests in Unity.

Installing Unity Test Framework

To install this package, follow the instructions in the Package Manager documentation.

Note: Search for the Test Framework package. In Unity 2019.2 and higher, you may need to enable the package before use.

Building and publishing

When you create a build of your application, you create a Player.

A Player is the platform-specific runtime application that Unity builds from your project.

You can control how Unity creates a build through Unity's build and player settings.

Topic	Description
Introduction to building	Understand the fundamentals of Unity's build process, including build modes, incremental building, and platform compatibility.
Content output of a build	Information about the files that Unity creates when you make a build of your project.
Create a build from the Editor	Use build profiles to build your application for different platforms with unique build configurations.
Create a clean build	Rebuild all content from scratch without using cached results to resolve build cache issues.
Create a scripts-only build	Build only the scripting assemblies without processing scenes and assets for faster iteration during development.
Customize the build pipeline	Create custom scripts and use callbacks to customize the build pipeline and run it from the command line.
Include additional files in a build	Use the StreamingAssets folder to include additional files in a build.
Reducing the file size of a build	Tips to reduce the size of the build.
Deterministic builds	Create consistent, reproducible builds by controlling factors that can cause build variations.
Analyze builds	Understand the build report and supporting files that Unity creates for each build to analyze and understand what your builds produce.

2D game perspectives reference

Game perspective	Description
Top-down	A 2D game that provides an overhead or bird's-eye view of the action.
Side-on	A 2D game where the character moves left or right (or up and down) and the screen scrolls with them. The perspective is from the side.
Isometric and 2.5D (three-quarter view)	2D games with isometric view simulate 3D geometry and a depth axis, but use an orthographic Camera instead of a perspective Camera. This gives the player a bird's-eye view of the action. For information on orthographic and perspective Cameras, see Cameras. 2.5D (also known as three-quarter view) games are 2D games that use 3D geometry for the environment and characters, but restrict the gameplay to two dimensions. The 3D effect has a visual rather than functional purpose.



2D game art style reference

Art style / technique	Description
Minimalist	Flat colors, clean lines and high readability, for example a mobile puzzle game.
Pixel art	Pixelated graphics, for example a retro gaming art style.
Illustrative	A cartoon, stylized or realistic art style.
Pre-rendered 3D	Use this technique to create 2D art with 3D tools. You create art that is rendered as Sprites or Sprite sequences. You can use this technique when you need multiple angles of an asset, for example for the 8- or 4-direction Sprite characters used in retro art.



Convert 2D assets from the Built-In Render Pipeline to URP

To convert 2D shaders, materials, and quality levels from the Built-In Render Pipeline to URP, do the following:

Go to Window > Rendering > Render Pipeline Converter.

Unity opens the Render Pipeline Converter window.

Set Source Pipeline to Built-in.

Set Target Pipeline to Universal Render Pipeline (2D Renderer).

In the Pipeline Converter tab, select the checkboxes next to Rendering Settings, Material Reference Converter, and Material Shader Converter.

If you have the 2D Pixel Perfect package installed in your project, also select the checkbox next to Pixel Perfect Camera Converter.

In the Pipeline Tools tab, select the checkboxes next to Parametric to Freeform Light Upgrade and Convert Built-in and URP (Universal Renderer) Materials to Mesh 2D-Lit-Default.

If you have Photoshop assets in your project, also select the checkbox next to Photoshop Assets Upgrader. For more information on these converters, refer to Render Pipeline Converter window for URP reference.

Select Scan.

Unity processes the assets in your project and displays the list of assets it can convert under your selected converters.

Select the checkboxes next to the assets you want to convert.

Select Convert Assets.

When Unity finishes the conversion process, the window displays the status of each conversion.